Retail Sales Analysis Project using MySQL

```sql

-- Create a 'Database':
create database retail_db;
use retail_db;

-- Create a 'Table':
CREATE TABLE retail_sales (
            transactions_id INT PRIMARY KEY,
            sale_date DATE,
            sale_time TIME,
            customer_id INT,
            gender     VARCHAR(10),
            age INT,
            category VARCHAR(15),
            quantiy INT,
            price_per_unit     FLOAT,
            cogs FLOAT,
            total_sale FLOAT
            );

select * from retail_sales
LIMIT 10;

select * from retail_sales
where quantiy = '';

-- SET SQL_SAFE_UPDATES = 0;

-- Deleted empty records using SQL Queries:
DELETE FROM retail_sales
WHERE quantiy = '' OR price_per_unit = '' OR cogs = '';

-- 3 records of Data deleted:
select distinct(count(transactions_id))
from retail_sales;

-- Data Exploration and Analysis:

-- Total sales recorded: '2001 Rows of Records'
select count(*) from retail_sales;

-- Total unique customers recorded: '155'
select count(distinct(customer_id))
from retail_sales;

-- Total categories recorded: '3'
select distinct(category) from retail_sales;
```

```sql

-- Analysing Key Business Questions(Q) & generating insights:

-- Q1 Retrieve all columns for sales made on '2024-11-05':
   -- 'Record of 11 columns retrieved'

select * from retail_sales
where sale_date = '2024-11-05';

-- Q2 Retrieve all transactions where category is 'Clothing' & qty sold is more than 3 in the month Nov 2024:

select * from retail_sales
where category = 'Clothing' AND quantiy > 3
AND sale_date between '2024-11-01' AND '2024-11-30';

-- Qty sold is more than 2 & less than 4 in the month of Aug 2025:

select * from retail_sales
where category = 'Electronics' AND quantiy = 3
AND sale_date between '2025-08-01' AND '2025-08-31';

-- Q3 Product category that generated highest total revenue: 'Electronics' has the higest rev of 3,14,370/-

SELECT category, sum(total_sale) AS total_revenue,
count(*) as total_orders
FROM retail_sales
GROUP BY category
ORDER BY total_revenue DESC;

-- Q4 The average 'Age' of customers who purchased items from 'Beauty' category: 'Average AGE' is 40

select round(avg(age)) AS avg_age from retail_sales
WHERE category = 'Beauty';

-- Q5 The monthly sales total for the year 2024 & 2025:
-- for 2024:
SELECT
    YEAR(sale_date) AS Sales_Year,
    MONTHNAME(sale_date) AS Month_Name,
    SUM(total_sale) AS Total_Sales
FROM retail_sales
WHERE YEAR(sale_date) = 2024
GROUP BY
    Sales_Year,
    Month_Name
ORDER BY
    Sales_Year,
    Month_Name;
```

```sql

-- for 2025:
SELECT
    YEAR(sale_date) AS Sales_Year,
    MONTHNAME(sale_date) AS Month_Name,
    SUM(total_sale) AS Total_Sales
FROM retail_sales
WHERE YEAR(sale_date) = 2025
GROUP BY
    Sales_Year,
    Month_Name
ORDER BY
    Sales_Year,
    Month_Name;

-- Q6 All transactions where the total_sale is greater than 1000:

select customer_id, gender, category, total_sale
from retail_sales
where total_sale > 1000;

-- Q7 The top 6 customers by lifetime spend: To find high value customers for loyalty programs

SELECT customer_id,
       SUM(total_sale) AS lifetime_spend,
       COUNT(*) AS trans_count
FROM retail_sales
GROUP BY customer_id
ORDER BY lifetime_spend DESC
LIMIT 6;

-- Q8 Total number of transactions made by each gender under each category:

select transactions_id, category, gender,
count(*) as Total_trans
from retail_sales
group by category, gender
order by 2;

-- Q9 Average order value (AOV) and 'Average Qty' per transaction by category:
      -- It helps compare basket size and value across categories.
SELECT category,
       round(AVG(total_sale),2) AS avg_order_value,
            round(avg(quantiy),1) AS avg_quantity
FROM retail_sales
GROUP BY category
ORDER BY avg_order_value DESC;

-- Q10 'Gross Margin' (GM) by category (revenue minus COGS): Reveals which categories are most profitable
```

```sql

SELECT category,
       SUM(total_sale) AS revenue,
       round(sum(cogs),1) AS total_cogs,
       round(SUM(total_sale) - SUM(cogs),2) AS gross_margin,
        round((SUM(total_sale) - SUM(cogs)) / (SUM(total_sale)) * 100,2) AS margin_pct
FROM retail_sales
GROUP BY category
ORDER BY gross_margin DESC;

-- Q11 The 'Avg Sale' for each months & Find out best selling month in each year: July 2024 and Feb 2025 has the highest

select * from
(
select
     YEAR (sale_date) as year_sale,
     MONTHNAME (sale_date) as month,
     ROUND (avg(total_sale),2) as avg_sales,
     RANK() OVER(PARTITION BY year(sale_date) order by round(avg(total_sale),2) desc) as rank_order
from retail_sales
group by 1, 2
)
AS Rank_table
where rank_order = 1;

-- Q12 'Days of week' which have the highest number of transactions: 'Tuesday' has the highest

SELECT DAYNAME(sale_date) AS weekday, COUNT(*) AS transactions
FROM retail_sales
GROUP BY weekday
ORDER BY transactions DESC;

-- Q13 The number of unique customers who purchased items from each category:

select category, count(distinct(customer_id)) as unique_cust
from retail_sales
group by 1;

-- Q14 Create each shift & number of sales (Morning <=12, Afternoon Between 12 & 17, Evening >17):
   -- Using CTE Functions

WITH hourly_sales_data
AS
(
select sale_time, customer_id, gender, category, total_sale,
     CASE
```

```sql
                WHEN hour(sale_time) < 12 THEN 'Morning'
                WHEN hour(sale_time) BETWEEN 12 AND 17 THEN 'Afternoon'
                ELSE 'Evening'
        END AS shift
        from retail_sales
        )
select shift, count(*) as total_orders
from hourly_sales_data
group by shift;

-- Q15 The hour of day that drives most revenue & transactions:
   -- This helps schedule promotions and staffing for peak hours.

SELECT HOUR(sale_time) AS sales_hr,
       COUNT(*) AS transactions,
       SUM(total_sale) AS revenue
FROM retail_sales
GROUP BY sales_hr
ORDER BY revenue DESC;

-- Q16 How does average spend differ by gender & age group? Insight:
    -- It supports targeted marketing and segmentation

SELECT gender,
       CASE
         WHEN age < 25 THEN 'Below 25'
         WHEN age BETWEEN 25 AND 34 THEN '25-34'
         WHEN age BETWEEN 35 AND 44 THEN '35-44'
         WHEN age BETWEEN 45 AND 54 THEN '45-54'
         WHEN age >= 55 THEN '55+'
       END AS age_group,
       round(AVG(total_sale),2) AS avg_spend
FROM retail_sales
GROUP BY gender, age_group
ORDER BY gender, age_group;

-- Q17 Which transactions show unusually low or negative margins:

SELECT transactions_id, sale_date, customer_id, category, quantiy,
price_per_unit, cogs,
       (total_sale) - (cogs) AS margin
FROM retail_sales
WHERE (total_sale) - (cogs) <= 0
ORDER BY margin ASC;

                        --- End of the SQL Analysis Project ---
```